Mittuniversitetet
MID SWEDEN UNIVERSITY

Design and Evaluation of an n8n-Based Workflow-to-Action Architecture for IoT-Style Fan Control

PC-hosted n8n with Python middleware, JSON contracts, and Raspberry Pi middleware validation

Yacoub Dawli

Bachelor Thesis
Main field of study: Computer Science
Credits: 15 hp
Semester, Year: Spring, 2026
Supervisor: Ali Hassan
Examiner: Patrik Österberg
Course code: DT099G
Programme: Master of Science in Engineering - Computer Engineering

At Mid Sweden University, it is possible to publish the thesis in full text in DiVA (see appendix for publishing conditions). The publication is open access, which means that the work will be freely available to read and download online. This increases the dissemination and visibility of the degree project.

Open access is becoming the norm for disseminating scientific information online. Mid Sweden University recommends both researchers and students to publish their work open access.

I/we allow publishing in full text (free available online, open access):

☒ Yes, I/we agree to the terms of publication.

☐ No, I/we do not accept that my independent work is published in the public interface in DiVA (only archiving in DiVA).

Sundsvall, 2026-06-12
Location and date

Civilingenjör i Datateknik
Programme/Course

Yacoub Dawli
Name (all authors names)

1999
Year of birth (all authors year of birth)

Abstract

This thesis designs, implements, and evaluates a PC-hosted, self-hosted proof-of-concept based on n8n for an IoT-style workflow-to-action pipeline. The scenario uses a temperature event to select a simulated fan-on or fan-off action. The objective is to examine whether workflow automation can be combined with a middleware boundary, structured action messages, validation design, and measurable evidence in a focused event-action system. The implementation uses Python middleware as the boundary between workflow logic and action execution, shared JSON contracts for sensor and action payloads, and a documented validation and safety design for allowed, malformed, and risky action outputs. The evaluation compares a deterministic n8n workflow based on a threshold rule with an agent-enhanced workflow that produces contract-shaped action output. Results were collected using CSV-based evaluation, with additional Raspberry Pi middleware validation where n8n remained on the PC while the middleware and action endpoint ran on the Raspberry Pi. The final successful local workflow runs produced the expected actions for the defined high- and low-temperature cases. The agent-enhanced workflow also produced contract-shaped fan-action output for the same cases, but with higher latency than the deterministic baseline. Raspberry Pi middleware validation showed that the PC-hosted n8n workflow could call the Pi-hosted middleware endpoint over the network and receive the expected simulated fan responses. The evaluation is limited to the defined cases and simulated fan actions, but it shows that the architecture can be implemented, measured, and discussed as a clear workflow-to-action proof-of-concept.

Keywords: Workflow automation, n8n, Internet of Things, middleware, JSON contracts, Raspberry Pi

Sammanfattning

Detta examensarbete utformar, implementerar och utvärderar ett PC-baserat, självhostat proof-of-concept baserat på n8n för en IoT-liknande kedja från arbetsflöde till handling. Scenariot använder en temperaturhändelse för att välja om en simulerad fläkt ska slås på eller av. Syftet är att undersöka om arbetsflödesautomatisering kan kombineras med ett mellanvarulager, strukturerade handlingsmeddelanden, valideringsdesign och mätbara resultat i ett fokuserat händelse- och handlingssystem. Implementationen använder Python-mellanvara som gräns mellan arbetsflödeslogik och handlingsutförande, gemensamma JSON-kontrakt för sensor- och handlingsmeddelanden samt en dokumenterad validerings- och säkerhetsdesign för tillåtna, felaktiga och riskfyllda handlingsutdata. Utvärderingen jämför ett deterministiskt n8n-arbetsflöde baserat på en tröskelregel med ett agentförstärkt arbetsflöde som producerar kontraktsformade handlingsutdata. Resultaten samlades in med CSV-baserad utvärdering och kompletterades med validering av Raspberry Pi-mellanvara, där n8n låg kvar på PC:n medan mellanvaran och handlingsslutpunkten kördes på Raspberry pi. De slutliga lyckade lokala arbetsflödeskörningarna gav förväntade handlingar för de definierade hög- och lågtemperaturfallen. Det agentförstärkta arbetsflödet producerade också kontraktsformade fläkthandlingar för samma fall, men med högre latens än den deterministiska baslinjen. Valideringen med Raspberry Pi visade att det PC-hostade n8n-arbetsflödet kunde anropa mellanvaruslutpunkten på Raspberry Pi över nätverket och få de förväntade simulerade fläktsvaren. Utvärderingen är begränsad till de definierade fallen och simulerade fläkthandlingar, men visar att arkitekturen kan implementeras, mätas och diskuteras som ett tydligt proof-of-concept.

Nyckelord: Arbetsflödesautomatisering, n8n, Internet of Things, mellanvara, JSON-kontrakt, Raspberry Pi

Acknowledgements/Foreword

This thesis marks the end of a challenging and valuable learning journey. I would like to thank my supervisor, Ali Hassan, for his guidance, feedback, and support throughout the work. I would also like to thank Stefan Forsström for the original project and research direction, which helped shape the topic of this thesis.

Finally, I want to thank my family for their patience, encouragement, and support during my studies. Their support has meant a great deal throughout this process.

Contents

Terminology/Notation

Table 1: Terminology used in the thesis.

Term	Meaning
Agent-enhanced workflow	The n8n workflow path where an LLM-based decision step produces a structured action output for the temperature-to-fan scenario.
API	Application Programming Interface. In this thesis, the API is the HTTP boundary exposed by the Python middleware.
CSV	Comma-separated values. The tabular file format used for raw and processed evaluation results.
Deterministic baseline	The rule-based n8n workflow path that selects `fan_on` or `fan_off` using a fixed temperature threshold.
GPIO	General-purpose input/output pins on hardware such as Raspberry Pi. Real GPIO fan control is outside the implemented scope.
HITL	Human-in-the-loop. A design idea where risky or unsupported action outputs require human review before execution.
IoT	Internet of Things. In this thesis, the term refers to the event-to-action pattern where a sensor-style event can lead to an action.
JSON contract	The shared JSON message structure that defines the expected shape of sensor events and action outputs.
LLM	Large language model. In this thesis, the LLM is used only in the agent-enhanced workflow decision step.
Middleware	The Python service that separates workflow decisions from action execution and exposes the simulated fan endpoints.
n8n	The self-hosted workflow automation engine used to build and run the deterministic and agent-enhanced workflows.
Raspberry Pi middleware validation	Validation where n8n remains on the PC while the Python middleware/action endpoint runs on the Raspberry Pi and is called over the network.
Webhook	An HTTP endpoint used by n8n to receive the temperature event or test input.

1 Introduction

This thesis investigates how a workflow automation system can be used as part of a measurable IoT-style action pipeline. The work focuses on a controlled "temperature to fan" scenario and examines whether a focused "workflow-to-action" architecture can be implemented and evaluated clearly without becoming a production IoT platform.

1.1 Background and Motivation

Workflow automation platforms are often used to connect triggers, decision logic, external services, and actions. In such systems, a workflow can receive an event, process it through configured nodes, and call an external endpoint. n8n is one such workflow automation platform and can be self-hosted for local experimentation and integration work [1], [2]. This makes it suitable for a thesis prototype where the workflow engine should be visible, configurable, and testable.

IoT systems commonly involve networked devices, sensing, and action-oriented interaction with the physical environment [3]. In this thesis, that idea is represented as an event-to-action pattern: a sensor or test event is received, a decision is made, and an action is triggered. Edge computing motivates placing parts of a system closer to the device or environment when latency, locality, or deployment context matters [4], [5]. Raspberry Pi is used in this thesis as a small edge-style validation device, while the workflow runtime remains PC-hosted [6].

This thesis uses a fan on/off scenario as a controlled proxy for that broader problem. The fan itself is simple, but the architecture around it is the important part: event handling, action selection, middleware execution, and recorded evidence are evaluated together. The scenario makes it possible to study a complete "event-to-action" path without expanding into a broad hardware or automation platform.

1.2 Problem Statement

The problem addressed in this thesis is that an IoT-style workflow is not sufficiently evaluated by showing only that a workflow can make a decision. If a workflow or agent-enhanced step can trigger an action, the system also needs a clear boundary between workflow logic and action execution. Without such a boundary, it becomes difficult to explain which component made the decision, which component executed the action, and what structure the action message was expected to follow.

The thesis treats the problem as an integration and evaluation problem. The system needs structured action messages, a documented validation design, a middleware layer that separates workflow decisions

from action execution, CSV-based evidence for result reporting, and limited hardware-side validation. JSON contracts are used to make the sensor and action payloads explicit [7]. The problem is investigated within a controlled proof-of-concept rather than through a large-scale deployment.

1.3 Aim

The aim of this thesis is to design, implement, and evaluate a self-hosted n8n-based proof-of-concept for a temperature-to-fan IoT-style action pipeline using Python middleware, structured contracts, lightweight evaluation, and Raspberry Pi middleware validation.

The aim is addressed by evaluating deterministic and agent-enhanced workflow behavior, contract-shaped action output, documented validation cases, CSV evidence, and Raspberry Pi middleware validation within the same proof-of-concept.

1.4 Research Questions

The thesis is guided by three research questions:

1. **RQ1:** How accurately does the deterministic n8n workflow produce the expected fan action for the defined temperature test cases?
2. **RQ2:** How accurately does the minimal agent-enhanced workflow produce contract-shaped fan-action output compared with the deterministic baseline for the defined cases?
3. **RQ3:** What latency, resource, and deployment behavior is observed during local execution and Raspberry Pi middleware validation?

Safety is treated as part of the contract and validation analysis rather than as a separate research question. In this report, *contract-shaped* means that the observed workflow or agent output follows the expected action fields and supported fan-action values for the defined cases. It should not be read as evidence of full runtime JSON Schema validation or production-grade safety enforcement. The role and limitations of the validation boundary are discussed further in Chapter 7.

1.5 Scope

The scope of this thesis is one measurable workflow-to-action pipeline. The implemented system uses a local PC-hosted n8n runtime, a Python middleware API, one temperature-event path, simulated `fan_on` and `fan_off` actions, a deterministic baseline workflow, a minimal agent-enhanced workflow, shared JSON contracts, a documented minimum

safety and validation design, a CSV-based evaluation harness, and Raspberry Pi middleware/action-endpoint validation.

The Raspberry Pi part is used to validate the middleware/action endpoint over the network, while the n8n workflow runtime remains on the PC. The thesis focuses on workflow behavior, contract-shaped action output, latency observations, and deployment evidence for the middleware boundary. Full n8n deployment on Raspberry Pi, physical GPIO-based fan control, production-grade runtime safety enforcement, broad multi-agent architecture, multiple models, multiple devices, and large-scale benchmarking are outside the scope.

1.6 Report Outline

Chapter 2 presents the theory and related work needed to understand workflow automation, IoT-style event-action systems, contracts, agent-enhanced workflows, validation, and edge hardware context. Chapter 3 describes the research approach and evaluation method. Chapter 4 explains the choice of approach and justifies the final system design. Chapter 5 describes the implemented proof-of-concept. Chapter 6 reports the observed results from the deterministic workflow, agent-enhanced workflow, evaluation harness, and Raspberry Pi validation. Chapter 7 interprets the results, answers the research questions, discusses limitations, and connects the fan scenario to broader use cases. Chapter 8 presents the final conclusions and future work. The appendix provides references to source code and supporting artifacts.

2 Theory and Related Work

This chapter presents the theoretical background and related work needed to understand the workflow-to-action architecture evaluated in this thesis. The chapter is organized around the conceptual chain used by the implemented system: a sensor or event input enters a workflow, the workflow makes either a deterministic or agent-enhanced decision, the decision is represented as a structured action object, the object is considered at a validation boundary, and the final action request is sent through a middleware/API layer to an IoT-style action endpoint.

The purpose of the chapter is not to survey all workflow engines, IoT platforms, edge devices, or language-model agent designs. Instead, it explains the concepts that support the defined evaluation scope and positions the thesis among related areas of research.

Figure 1 summarizes this conceptual event-to-action chain.

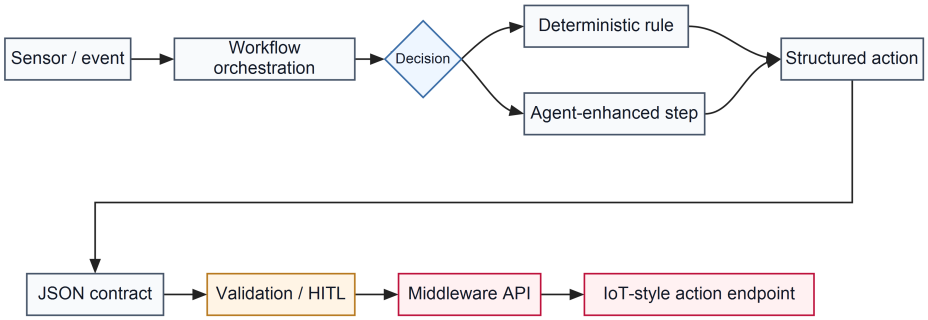

Figure 1: *Conceptual event-to-action pattern used to frame the thesis.*

2.1 Workflow Automation and Orchestration

Workflow automation concerns the coordination of tasks, decisions, and integrations through an explicit process structure. A workflow can start from an event, apply decision logic, transform data, and call external systems. Workflow automation is therefore relevant for event-action systems, where the goal is to connect an incoming event to a controlled response rather than to perform isolated computation. Workflow patterns provide a research basis for describing recurring control-flow structures such as branching, routing, and synchronization [2].

In this thesis, workflow automation is used as the orchestration layer. The workflow engine receives a temperature event, evaluates it, and

calls middleware endpoints. n8n is the concrete platform used for this purpose; it provides a self-hostable workflow environment with trigger nodes, data-handling nodes, and HTTP integration nodes [1]. The thesis does not compare n8n against other workflow platforms. It uses n8n as the practical workflow engine in a controlled implementation where workflow behavior can be evaluated.

2.2 IoT Event-Action Systems and Edge Computing

IoT systems commonly connect sensors, software services, networks, and actions in a physical or operational environment. A sensor event may represent a temperature reading, a state change, or another observation that requires a response. The response may be an alert, a control action, or a decision to do nothing. This event-action pattern is central to the thesis scenario, where a temperature event is used to select a simulated fan action. General IoT research describes sensing, communication, processing, and actuation as core parts of Internet of Things systems [3].

Edge computing becomes relevant when processing or action endpoints are moved closer to devices or operational environments instead of relying entirely on remote infrastructure. Edge computing can support latency-sensitive behavior and local context handling, and edge-intelligence research connects this placement of computation to AI-related processing closer to devices [4], [5]. Raspberry Pi is relevant in this context because it is a compact computing platform that can run software services and connect to hardware interfaces [6]. In this thesis, Raspberry Pi is used for middleware/action-endpoint validation while the workflow engine remains PC-hosted. The evaluation stays focused on the workflow-to-action boundary rather than on a complete edge deployment.

2.3 Middleware and API Boundaries

Middleware is useful when decision logic should be separated from action execution. In a workflow-to-action architecture, the workflow should not automatically become the hardware-facing control component. A middleware/API layer can receive a structured action request, expose a defined endpoint set, and isolate the workflow from the internal details of the action implementation. The separation makes the system easier to analyze because workflow logic, structured contracts, validation design, and action execution can be discussed as distinct parts of the architecture.

REST and HTTP provide established architectural and communication

concepts for networked systems [8], [9]. In this thesis, the middleware/API boundary is used in a focused form: workflow output is routed to middleware endpoints rather than directly to a physical actuator. This boundary is also what allows the same action pipeline to be used for both deterministic workflow output and minimal agent-enhanced output.

2.4 Structured Data, JSON Contracts, and Validation

Structured data formats reduce ambiguity between connected components. If a workflow emits free text, the receiving system must infer the intended action. If the workflow emits a structured object, the receiving system can inspect fields, allowed values, and missing data against an explicit structure. This is why structured sensor and action messages matter in a workflow-to-action architecture.

JSON Schema provides a vocabulary for describing and validating the structure of JSON data [7]. A schema can define required fields, allowed values, and whether unexpected fields are permitted. In this thesis, structured contracts describe the expected shape of sensor events and action outputs. The contracts support predictable routing and provide a basis for discussing validation. They do not by themselves create production-grade safety enforcement, but they make the expected action boundary explicit.

2.5 Deterministic Workflows

A deterministic workflow uses fixed logic to map inputs to outputs. Given the same input and the same workflow configuration, the expected decision path is the same. In the thesis scenario, the deterministic baseline uses a temperature threshold: a value at or above the threshold leads to `fan_on`, while a lower value leads to `fan_off`.

Deterministic workflows are useful as baselines because their behavior is transparent and inspectable. The rule can be stated directly, the expected output can be defined before execution, and deviations can be identified clearly. For this thesis, the deterministic path establishes a reference point for evaluating action correctness and latency observations before considering the agent-enhanced path.

2.6 Minimal Agent-Enhanced Workflows

An agent-enhanced workflow includes a decision step where a language model contributes to selecting or producing an action. In broader research, language-model agents and agentic AI are discussed as systems that connect model output to reasoning, tool use, and action-oriented tasks [10], [11], [12]. The thesis uses a controlled version of

this idea: one language-model decision step is placed inside a workflow and is asked to produce structured action output for the same temperature-to-fan task as the deterministic baseline.

The relevant theoretical issue is whether an LLM-based workflow step can be constrained so that its output fits a shared action contract and can be routed through the same middleware boundary as a rule-based decision. This framing keeps the agent-enhanced path connected to the workflow-to-action architecture rather than treating it as a general multi-agent system.

2.7 Safety, Validation, and Human-in-the-Loop Concepts

When workflow or agent output may trigger an action, validation becomes part of the architecture. A system should be able to distinguish between an allowed action, a malformed action, and a risky or unsupported action. In the thesis scenario, allowed actions are restricted to the simulated fan actions, while malformed or unknown actions should not be treated as valid action requests.

Human-in-the-loop concepts are relevant when automated output may require review before execution. Human involvement can help with cases where output is uncertain, risky, or outside the expected action set. Research on interactive machine learning describes the role of humans in guiding and controlling machine-assisted systems [13]. In this thesis, the safety material is a documented validation and approval design. It provides material for discussing allowed, blocked, and risky cases, without being presented as production-grade runtime safety enforcement.

2.8 Related Work and Positioning

This thesis draws on several related research areas rather than belonging to only one of them. IoT and edge computing research explains why event-driven systems and local action endpoints are relevant for systems that connect sensor input to operational responses [3], [4]. Workflow research describes how explicit process structures can coordinate triggers, decisions, routing, and service calls [2]. Research on language-model agents shows how LLMs can be connected to action-oriented tasks and tool use, while this thesis applies that idea in a single agent-enhanced workflow step [10].

Structured contracts and API boundaries connect these areas at the system-design level. JSON Schema provides a way to describe expected message structures [7], while REST and HTTP concepts support the separation between workflow logic and networked action end-

points [8], [9]. Human-in-the-loop and validation concepts further support the idea that action-producing systems should not treat every generated output as directly executable [13].

The temperature-to-fan scenario is intentionally simple, but the architectural pattern is relevant to broader application areas. Smart-building research discusses IoT architectures for building-management contexts [14], and industrial IoT research discusses connected monitoring and cyber-physical systems in operational environments [15]. These areas are not implemented in this thesis, but they show why a workflow-to-action pipeline can be useful beyond the fan example.

The gap addressed in this thesis is the practical integration of these concepts in one measurable implementation. Existing work discusses IoT systems, edge computing, workflow orchestration, and agent-based tool use, but this thesis evaluates how these ideas can be combined in a controlled workflow-to-action architecture. The implemented system connects a self-hosted workflow engine, structured action output, middleware/API boundaries, validation concepts, CSV-based measurement, and Raspberry Pi middleware/action-endpoint validation in one temperature-to-fan scenario. The contribution is an evaluated integration that can be inspected, reproduced, and extended in future work.

2.9 Summary of Theory

The theory in this chapter prepares the reader for the methodology and system design chapters. Workflow automation explains the orchestration layer, IoT and edge computing explain the event-action context, middleware and API boundaries explain the separation between decisions and actions, structured contracts explain the message boundary, and validation/HITL concepts explain why generated or workflow-produced actions need constraints before execution. The following chapters use this background to define how the proof-of-concept was developed, why the final architecture was chosen, and how its behavior was evaluated.

3 Methodology

This chapter describes how the thesis work was carried out and how the proof-of-concept system was evaluated. The method was chosen to match the defined proof-of-concept scope: one controlled temperature-to-fan scenario, two workflow decision modes, lightweight measurement, and Raspberry Pi middleware/action-endpoint validation.

3.1 Research Approach

The thesis uses a design-and-evaluate proof-of-concept approach. The work first defines a limited technical problem, then builds an artifact that addresses that problem, evaluates the artifact with defined test cases, and finally discusses the limitations of the evidence. The approach fits the thesis because the research questions are not answered only by theoretical comparison; they require an implemented system that can produce observable workflow behavior, action outputs, latency values, resource observations, and deployment evidence [16], [17].

The evaluated artifact is a focused workflow-to-action system with PC-hosted n8n, middleware-based action execution, structured contracts, and file-based evaluation evidence. The evaluation is therefore artifact-centered and scenario-based rather than a benchmark of many platforms, devices, or language models.

3.2 Development Method

The implementation was developed through controlled stages. This staged method was used to reduce scope risk and keep each part of the system testable before adding the next one. The work first defined the evaluation scope, then implemented the runtime setup, middleware boundary, shared contracts, workflow modes, validation design, evaluation harness, and Raspberry Pi middleware validation.

Each stage introduced one necessary layer and produced evidence before the next layer was added. The runtime setup established the workflow execution environment. The middleware provided the action boundary. The shared contracts made the event and action payloads explicit. The workflow modes provided the deterministic and agent-enhanced comparison. The evaluation harness made the system measurable through CSV files, and the Raspberry Pi validation tested the middleware/action endpoint on separate hardware.

Once the evaluated system had produced enough evidence for the research questions, the implementation was kept stable during result reporting. Larger extensions, such as deeper Raspberry Pi deployment, physical hardware control, broader agent behavior, and larger-scale

evaluation, were kept outside the evaluated implementation and are treated as future work.

3.3 Evaluation Design

The evaluation was designed around one temperature-to-fan scenario. The same input cases were used for the deterministic baseline workflow and the minimal agent-enhanced workflow so that the two modes could be compared on the same task. The evaluation also included documented safety and validation cases and a separate Raspberry Pi validation setup.

The deterministic baseline was evaluated by checking whether the fixed n8n threshold rule produced the expected fan action for the defined high- and low-temperature cases. The minimal agent-enhanced workflow was evaluated by checking whether the agent path produced the expected fan action and whether the observed output was contract-shaped, meaning that it contained the expected action fields and supported fan-action values for the defined cases. This check is distinguished from full runtime JSON Schema validation. The evaluation did not include a measured runtime schema-validation gate or logged pass/fail validation results for every action output. The safety and validation cases were evaluated as documented design cases: one allowed action, one malformed blocked action, and one risky or unsupported action that should not be executed directly. The Raspberry Pi validation evaluated whether PC-hosted n8n could call a Raspberry Pi-hosted middleware/action endpoint for the same temperature-to-fan scenario.

The defined high- and low-temperature workflow cases used in the evaluation are listed in Table 2. Full repository paths for the dataset, scripts, and result files are listed in Appendix A.

Table 2: Defined workflow test cases.

Test case type	Input temperature	Expected action	Mode
High-temperature workflow case	31.4 C	`fan_on`	Deterministic and agent
Low-temperature workflow case	24.5 C	`fan_off`	Deterministic and agent

The safety cases in the dataset use action objects rather than temperature events. The allowed case contains a complete `fan_on` action for `fan_1`. The blocked case omits required fields. The risky case uses

an unsupported `open_window` action and an unknown target. These safety cases provide design evidence for the documented validation boundary rather than proof of production runtime enforcement.

3.4 Metrics and Data Collection

The evaluation records both behavioral and measurement-oriented fields. The behavioral fields include the expected action, observed action, success or failure, expected safety result, and observed safety result where applicable. The measurement fields include latency, CPU values where available, RAM values where available, and thermal observations where available. Contract-shaped output is considered where workflow or agent output is expected to follow the shared action object structure. This means that the observed output is checked against the expected fields and supported values at the report-evidence level, not that a full runtime JSON Schema validator was implemented and measured.

Data collection is CSV-based. Raw result rows and processed summaries are written to the evaluation result folders listed in Appendix A. The collection script posts test payloads to the configured workflow webhook and records the raw result fields. For workflow modes, success is recorded when the HTTP request succeeds and the observed action matches the expected action. The script measures latency using Python timing around the webhook HTTP request. It also attempts best-effort CPU and RAM collection through Docker statistics when available [18]. Thermal values are not fabricated; when no supported thermal source is available, the field is recorded as `not_available`.

The aggregation script aggregates raw CSV files into processed latency, resource, safety-outcome, and baseline-versus-agent summaries. CSV is used as a simple tabular exchange format, with Python's standard `csv` module used by the scripts [19], [20]. These files support the Results chapter by separating raw observations from summarized comparison tables. The evaluation protocol also notes that safety-mode rows can document expected safety behavior without proving runtime enforcement unless enforcement logic is implemented and logged.

3.5 Raspberry Pi Validation Method

Raspberry Pi validation was used to test the middleware/action endpoint on separate hardware, while n8n remained running on the PC. The PC-hosted n8n workflow called the Raspberry Pi middleware over HTTP.

The Raspberry Pi validation checks whether the middleware/action endpoint can run on the Raspberry Pi and be called over the network

while the n8n workflow remains PC-hosted. In this setup, the Raspberry Pi represents the action-endpoint side of the architecture rather than the full workflow runtime. The fan action is simulated, so the validation focuses on networked middleware behavior and deployment observations rather than physical GPIO-based fan control.

The Raspberry Pi evidence was stored with the evaluation artifacts listed in Appendix A. The method included the same high- and low-temperature fan cases, full workflow latency collection for the PC-to-Pi workflow path, supporting direct endpoint latency observations, CPU/RAM observations for the middleware process, and thermal observation from the Raspberry Pi evidence.

3.6 Validity and Limitations of the Method

The evaluation method is designed for a controlled proof-of-concept rather than broad benchmarking. The number of test cases is small because the thesis focuses on one temperature-to-fan scenario and compares the defined deterministic and agent-enhanced workflow paths under the same inputs. The results are evidence for the implemented workflow-to-action pipeline, not general performance claims for all n8n, IoT, or agentic AI systems.

The Raspberry Pi validation focuses on the middleware/action-endpoint side of the architecture while the n8n workflow remains PC-hosted. The fan action is simulated, so the method evaluates workflow behavior, contract-shaped action output, middleware routing, latency observations, and deployment evidence rather than physical GPIO-based fan control or production runtime safety. Full runtime schema validation and safety enforcement are outside the measured implementation.

3.7 Summary of Method

The method supports the research questions by combining a controlled implementation artifact with repeatable, file-based evaluation evidence. It compares the two workflow decision modes under the same temperature cases, records action outcomes and measurements where available, and uses Raspberry Pi middleware validation as separate-hardware deployment evidence.

4 Choice of Approach / System Design

This chapter explains the design choices that shaped the proof-of-concept before the implementation details are presented. The purpose is to justify the selected workflow-to-action architecture, including the PC-hosted n8n runtime, Python middleware boundary, deterministic and agent-enhanced decision paths, validation design, and Raspberry Pi middleware validation.

4.1 Design Requirements and Constraints

The system design was constrained by the need to produce a measurable and finishable thesis artifact. The central requirement was one complete event-to-action path that could be implemented, tested, and evaluated within the available thesis time frame. For this reason, the design focuses on a controlled temperature-to-fan scenario rather than multiple sensors, multiple actuators, or a general-purpose IoT platform.

The system also needed a clear separation between workflow logic and action execution. The workflow engine could decide which action should be taken, but it should not be treated as a direct hardware controller. This led to a design where n8n handles workflow orchestration, while Python middleware exposes the API boundary for action execution. This separation improves explainability because the report can distinguish event handling, decision logic, validation, middleware behavior, and simulated action execution.

Another requirement was structured communication between the workflow side and the action side. The design therefore uses JSON contracts for sensor events and action outputs instead of relying on loose text. The contracts support predictable routing, validation, and evaluation. The evaluation itself also needed to produce evidence that could be inspected and reused in the report, so lightweight CSV files were chosen as the main result format. Raspberry Pi validation was included to support the edge-computing angle: it tests the middleware/action endpoint on separate hardware while avoiding a full Raspberry Pi deployment project.

4.2 Alternatives Considered

Several broader alternatives were considered but not selected because they would have shifted the focus of the thesis. A full n8n deployment on Raspberry Pi would have made the project more hardware- and deployment-oriented. That alternative may be valuable in future work, but it would also introduce operating-system, container,

storage, and performance variables that are not necessary for answering the current research questions [21]. The selected alternative keeps n8n on the PC and validates only the middleware/action endpoint on Raspberry Pi hardware.

Another alternative was to let workflow nodes call hardware-facing actions directly. This was rejected because it would blur the safety and contract boundary. A middleware/API boundary makes the action path explicit and gives the system a single place where workflow output can be translated into controlled action execution. It also keeps future hardware integration possible without requiring workflow nodes or agent output to become direct device-control mechanisms.

The workflow design also required a choice between a deterministic-only system and a system with both deterministic and agent-enhanced paths. A deterministic-only workflow would be easier to explain and measure, but it would not test whether an LLM decision step can produce structured action output for the same event-action task. The chosen design therefore includes both: a deterministic threshold baseline and a minimal agent-enhanced workflow. The agent path is intentionally small and is not used as a basis for model benchmarking or broad multi-agent behavior.

For workflow output, loose natural-language text was rejected in favor of a shared JSON contract. Free-form text would make routing, validation, and comparison harder to report objectively. A contract-shaped action object gives the system clear expected fields and allowed values. Similarly, a full runtime safety enforcement system was outside the defined implementation. Instead the report uses a documented validation design with allowed, blocked, and risky cases. That is enough to describe the intended safety boundary while keeping clear that production-grade enforcement is not implemented.

Real GPIO fan control was also considered outside the current scope. Physical hardware would add electrical and device-specific variables that are not needed to evaluate the workflow, middleware, contract, validation, and measurement pipeline. The chosen simulated fan endpoint keeps the test controlled. Finally, a heavy monitoring stack such as Prometheus or Grafana was rejected in favor of CSV-based evaluation scripts. CSV output is less sophisticated, but it is transparent, repeatable, and proportionate to the scale of the thesis.

4.3 Chosen System Design

The chosen architecture is a PC-hosted n8n workflow system with a Python middleware boundary and lightweight evaluation support. n8n runs on the PC using Docker. The evaluation setup invokes one of

two separate PC-hosted n8n workflow modes for a given run. In the deterministic baseline mode, the workflow applies a fixed threshold rule. In the minimal agent-enhanced mode, a separate workflow produces structured action output that is expected to match the shared action contract. The selected evaluation mode therefore determines which workflow is executed, rather than a single workflow dynamically choosing between modes.

The shared JSON contracts define the sensor-event and action-output boundary. The documented validation design describes which outputs are allowed, which malformed outputs should be blocked, and which risky or unknown outputs should not be executed directly. The Python middleware exposes the API/action endpoints used by the workflows. The evaluation harness records raw and processed CSV files so that workflow behavior, latency, and resource observations can be reported.

Limited Raspberry Pi validation is included by moving the middleware/action endpoint to Raspberry Pi hardware while n8n remains running on the PC. This design keeps the workflow engine stable and repeatable while still testing whether the PC-hosted workflow can call an edge-side endpoint over the network.

The resulting report-level architecture is shown in Figure 2.

4.4 Rationale for PC-Hosted n8n and Raspberry Pi Middleware Validation

PC-hosted n8n was chosen for stability, repeatability, and reduced deployment risk. The local Docker-based setup provides a repeatable workflow runtime and avoids making the thesis dependent on whether n8n itself can be optimized for Raspberry Pi hardware. The workflow comparison can then focus on behavior and measurement rather than on platform-specific installation issues [1], [22].

The Raspberry Pi validation was included to test the action-endpoint side of the architecture on separate hardware. In this design, the PC-hosted workflow sends requests over the network to a Raspberry Pi-hosted middleware/action endpoint. This preserves the main workflow setup while still giving evidence about edge-style middleware deployment and networked action handling [3], [4], [6].

The chosen design therefore separates workflow orchestration from edge-side action execution. Running the full n8n workflow runtime on Raspberry Pi would introduce a different deployment and resource question, so that extension is treated as future work rather than as part of the evaluated design.

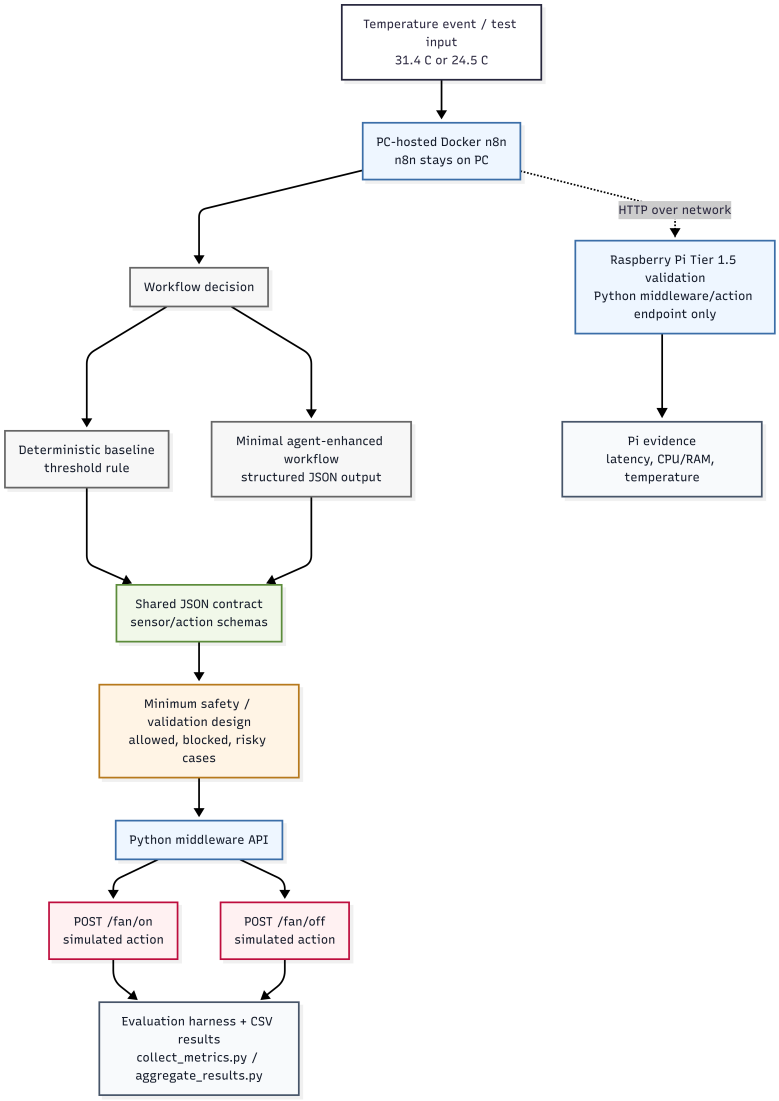

Figure 2: *Report-level architecture of the implemented workflow-to-action system.*

4.5 Rationale for Deterministic and Agent-Enhanced Paths

The deterministic path provides a predictable baseline. It uses a fixed temperature threshold and gives the evaluation a simple comparison point: for the defined high-temperature case, the expected action is `fan_on`; for the defined low-temperature case, the expected action is `fan_off`. This baseline separates expected workflow correctness from the additional uncertainty introduced by an LLM decision step.

The minimal agent-enhanced path was selected to test whether an LLM decision step can produce contract-shaped action output for the same controlled event-action task. The agent path uses one prompt set, one memory choice, and the same middleware endpoints as the deterministic baseline. This makes the comparison focused on structured action output and observed behavior for the defined temperature cases.

4.6 Rationale for Middleware, Contracts, and Validation

The middleware boundary was chosen because workflow or agent output should not be treated as direct hardware control. By routing actions through Python middleware, the system has a clear API layer between decision logic and action execution. This makes it possible to describe the action path precisely and to keep future hardware integration separate from the n8n workflow design.

The JSON contracts make this boundary explicit. The sensor-event contract defines the input structure, and the action contract defines the allowed output structure. Contract-shaped messages make the system easier to inspect and evaluate because the report can compare expected fields, allowed actions, and observed routing behavior [7].

The documented validation design adds a safety-oriented boundary around the action contract. It defines allowed output, malformed output, and risky or unknown output. The design improves explainability and reportability, but it should not be confused with a fully enforced safety system. The implemented thesis artifact contains a documented validation design, not production-grade runtime safety enforcement.

4.7 Rationale for Simulated Fan Actions

The fan action was simulated to keep the proof-of-concept controlled. The relevant thesis problem is not whether a physical fan can be switched on or off; it is whether a workflow can receive an event, produce or

route a contract-shaped action object, pass through a documented boundary, call middleware, and produce measurable evidence. Simulated `fan_on` and `fan_off` endpoints allow those parts of the system to be evaluated without introducing physical wiring, GPIO libraries, electrical reliability, or actuator-specific behavior.

No real GPIO hardware was controlled in the chosen design. This is a limitation, but it is also what keeps the implementation focused on workflow orchestration, middleware design, contracts, validation, and measurement. Future work could replace the simulated endpoint with physical GPIO or actuator hardware while preserving the same overall boundary between workflow logic and action execution.

4.8 Rationale for Lightweight Evaluation

The evaluation design uses CSV files because they are transparent, easy to inspect, and sufficient for the scale of the thesis. The collection script records raw observations, and the aggregation script produces processed summaries. This gives the report concrete evidence without requiring a large monitoring stack.

The scripts `collect_metrics.py` and `aggregate_results.py` support repeatable measurement of action outcomes, latency, and resource values where available. This lightweight approach fits the scale of the thesis better than a full observability platform. The evaluation is thesis-scoped and small-scale; it is not designed as large-scale benchmarking across devices, models, or deployment environments.

4.9 Summary of Chosen Approach

The chosen approach gives the thesis a complete workflow-to-action proof-of-concept while keeping the system focused and explainable. PC-hosted n8n provides the workflow runtime, Python middleware provides the action boundary, shared JSON contracts make the event and action structures explicit, documented validation cases describe the safety boundary, simulated fan endpoints keep the scenario controlled, CSV files provide measurable evidence, and Raspberry Pi middleware validation adds separate-hardware evidence without changing the core architecture.

5 Implementation

This chapter describes the implemented proof-of-concept system. The implementation follows the defined Raspberry Pi middleware/action-endpoint validation scope for the thesis: one temperature-event path, two workflow decision modes, one shared action contract, one documented minimum safety and validation design, simulated fan actions, CSV-based evaluation, and Raspberry Pi validation limited to the middleware/action endpoint side. The implementation therefore focuses on a complete and measurable workflow-to-action slice that can be evaluated against the research questions.

5.1 System Overview

The overall architecture was introduced in Chapter 4 and shown in Figure 2. This chapter focuses on how the implemented components were realized and connected.

The implemented system follows a temperature-event-to-action pipeline. A test input enters one of the two evaluated PC-hosted n8n workflow modes. In the deterministic baseline run, n8n applies the fixed temperature-threshold rule. In the agent-enhanced run, a separate minimal agent-enhanced workflow produces structured action output for the same defined temperature cases. The selected mode therefore determines which workflow is executed; the workflow does not dynamically choose between the deterministic and agent-enhanced modes at runtime.

In both modes, the resulting action is represented through the shared contract and documented validation boundary, and the Python middleware routes the request to a simulated `fan_on` or `fan_off` action. The evaluation harness then records the resulting behavior as CSV evidence.

The following sections describe the local n8n runtime, the Python middleware API, the shared contract layer, the deterministic and agent-enhanced workflows, the documented validation design, the evaluation harness, and the Raspberry Pi middleware/action-endpoint validation.

Full repository paths for configuration files, schemas, workflow exports, scripts, and evidence files are listed in Appendix A.

5.2 Local n8n Runtime with Docker

The workflow engine was implemented as a local self-hosted n8n runtime on the PC. The runtime is defined by the Docker Compose configuration and uses the pinned container image `n8nio/n8n:1.123.37`. The compose file exposes n8n on port `5678`, persists n8n data in a

Docker volume, and uses local development settings such as disabled secure cookies and disabled diagnostics/version notifications [1], [22].

Figure 3 shows the local Docker Desktop runtime with the n8n container running. The log output indicates that n8n was ready on port 5678, which supports the PC-hosted workflow runtime used in the implementation.

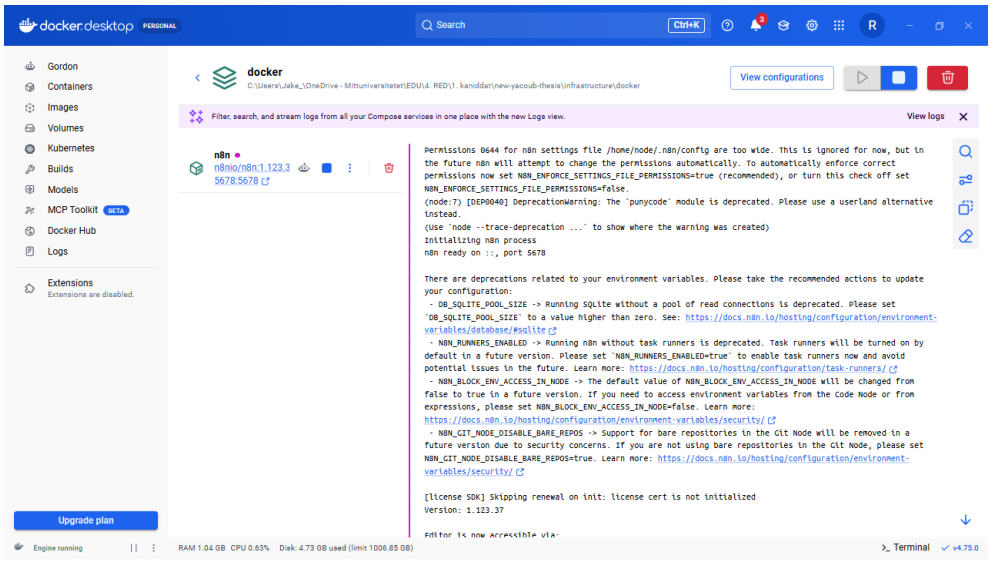

Figure 3: *Docker Desktop showing the local n8n container running and ready on port 5678.*

The PC-hosted Docker setup was chosen for stability and repeatability. It allowed the workflow engine, middleware, contracts, and evaluation harness to be developed in a controlled local environment before adding Raspberry Pi validation. This also reduced the risk of turning the thesis into a deployment study of n8n on constrained hardware. Full n8n deployment on the Raspberry Pi was intentionally outside the implemented scope. The Raspberry Pi validation therefore appears later in the implementation as a split validation of the middleware/action endpoint side, not as a replacement for the PC-hosted n8n baseline.

The local n8n runtime is the basis for both implemented workflow modes. The deterministic baseline and the minimal agent-enhanced workflow both receive temperature events through n8n webhook flows and route their resulting fan action to the middleware. Keeping the workflow engine constant makes the two workflow modes easier to compare in the evaluation chapter.

Runtime verification evidence for the local n8n setup is listed in Appendix A.

5.3 Python Middleware API Bridge

The Python middleware implements the boundary between workflow decisions and action execution. It can be started with `python -m middleware.api.app`. The HTTP request handling uses Python standard-library HTTP server components [23], while the simulated sensor and fan state are implemented in the middleware support files.

Figure 4 shows the Python middleware started on the Raspberry Pi through an SSH session. The middleware is bound to `0.0.0.0:8000`, which makes it suitable for network-based validation from the PC-hosted workflow.

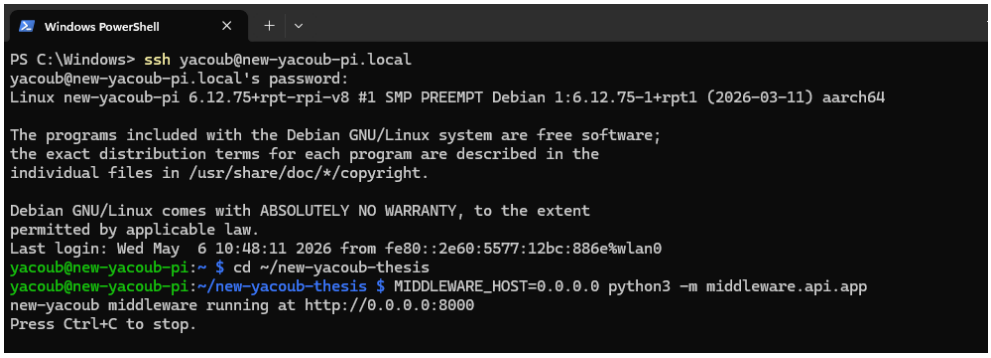

Figure 4: *Python middleware started on the Raspberry Pi through SSH and bound to port 8000.*

The middleware exposes four implemented endpoints. `GET /status` returns the middleware status and the current simulated hardware state. `POST /sensor-event` accepts a temperature event, normalizes it with a minimal inline check, stores it as the latest sensor event, and can forward it to n8n when a webhook URL is configured. `POST /fan/on` changes the simulated fan state to `on`, and `POST /fan/off` changes the simulated fan state to `off`. The fan endpoints return JSON responses that include the action name, the resulting fan state, a `simulated` flag, and a reason field.

This middleware boundary is important because it prevents workflow nodes from being treated as direct hardware controllers. Even in the Raspberry Pi validation, the action remains a simulated middleware action. No GPIO library, real fan driver, or physical actuator path is implemented in the final implementation. This keeps the implementation aligned with the thesis scope: the work evaluates an IoT-style

action pipeline and contract boundary, not physical electronics control.

5.4 Shared JSON Contracts

The shared interface layer defines the data structures that connect the middleware side and the workflow/agent side. The main reason for using structured JSON contracts was to avoid loose text as an action boundary. A free-form agent response or informal workflow payload would make routing and validation ambiguous. A small schema-based contract, by contrast, makes the expected fields explicit and gives the safety design a precise object to check [7].

The input-side schema defines the temperature sensor event used by the thesis scenario. It requires `sensor_id`, `timestamp`, `type`, `value`, and `unit`. The supported sensor type is `temperature`, and the supported unit is `C`. This intentionally small input contract keeps the scenario measurable and avoids expanding into multiple sensors or device families.

The output-side schema defines the fan action object that workflow-produced actions should follow. It requires `action_id`, `target`, `reason`, and `requires_approval`. The allowed action identifiers are `fan_on` and `fan_off`, and the allowed target is `fan_1`. The schema also disallows additional properties. These restrictions define the intended contract boundary for predictable routing: a supported `fan_on` action maps to `POST /fan/on`, and a supported `fan_off` action maps to `POST /fan/off`. In the evaluated implementation, the workflow output was parsed and routed for supported fan actions, but full runtime JSON Schema validation with logged pass/fail validation results was not implemented.

The repository also includes example payloads for a valid sensor event, a valid fan-on action, and a blocked action example. These examples support reportability and later evaluation, but they should not be interpreted as proof of runtime schema validation or production safety enforcement. The contract layer defines the expected boundary; the safety layer documents how that boundary should be checked in a stronger implementation.

5.5 Deterministic Baseline Workflow

The deterministic baseline workflow is implemented in the deterministic workflow files. It is the non-AI comparison anchor for the thesis. The workflow receives a temperature event through an n8n Webhook node, reads the event value from the parsed request body, applies one threshold rule, and then calls one of the simulated fan endpoints.

The rule is intentionally simple:

> if temperature $\geq$ 30.0 C, request `fan_on`; otherwise, request `fan_off`.

The value `30.0 C` is the decision threshold used by the deterministic workflow. The evaluation values `31.4 C` and `24.5 C` are not additional thresholds; they are the two selected test inputs used to exercise the two branches of this rule. The high-temperature input `31.4 C` is above the threshold and is expected to route to `fan_on`, while the low-temperature input `24.5 C` is below the threshold and is expected to route to `fan_off`.

When the temperature is at or above the threshold, the workflow creates an action object corresponding to `fan_on` and calls the local `POST /fan/on` endpoint. When the temperature is below the threshold, it creates an action object corresponding to `fan_off` and calls the local `POST /fan/off` endpoint. The host-Docker bridge address is used because n8n runs inside Docker while the middleware runs on the host during local PC execution.

The deterministic workflow exists for two reasons. First, it provides a predictable reference behavior for the same temperature-to-fan task used by the agent-enhanced workflow. Second, it gives the evaluation a simple baseline against which action correctness and latency observations can be compared. The baseline does not use prompts, memory, model calls, safety approval nodes, GPIO, or Raspberry Pi deployment.

The deterministic n8n workflow canvas is shown in Figure 5.

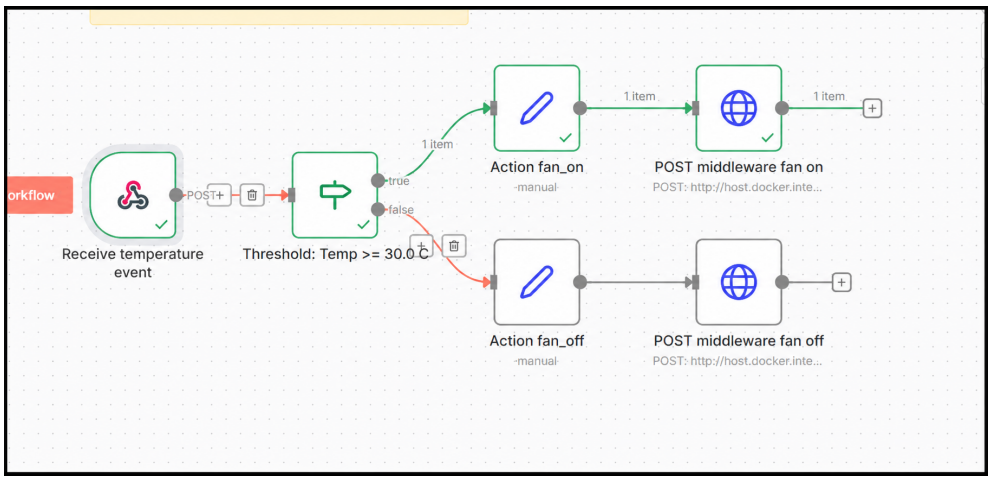

Figure 5: *Deterministic baseline n8n workflow canvas. The IF node label shows the fixed decision threshold,* `Temp` $\geq$ `30.0 C`*; the evaluated inputs* `31.4 C` *and* `24.5 C` *are test cases chosen above and below this threshold.*

5.6 Agent-Enhanced Workflow

The minimal agent-enhanced workflow is implemented in the agent workflow files. It uses the same temperature-event scenario and the same middleware fan endpoints as the deterministic baseline, but it replaces the fixed threshold decision node with a constrained LLM decision step.

The workflow receives the temperature event through a Webhook node, prepares the current event as agent input, sends it to one LLM decision step, parses the returned action object, and routes supported fan actions to the middleware. The Step 7 evidence identifies the configured model node as Google Gemini Chat Model. The workflow uses one prompt set and one documented memory choice. The chosen memory strategy is stateless execution with no memory node connected.

The prompt constrains the LLM to return a structured JSON action object rather than a free-form explanation. The expected output follows the same action contract used elsewhere in the thesis: `action_id`, `target`, `reason`, and `requires_approval`. The workflow then parses this structured output and routes supported `fan_on` actions to `POST /fan/on` and supported `fan_off` actions to `POST /fan/off`. A minimal unrouted branch exists for output outside the expected action values, but this branch is only a workflow-level routing fallback. It should not be described as full runtime JSON Schema validation or as the formal safety layer.

The purpose of this workflow is focused. It tests whether an agent-style decision step can produce contract-shaped structured action out-

put for the same temperature-to-fan task as the deterministic baseline. The workflow is configured with one prompt setup, one model path, and stateless execution so that its behavior can be compared directly with the deterministic path. The agent-enhanced workflow canvas is shown in Figure 6.

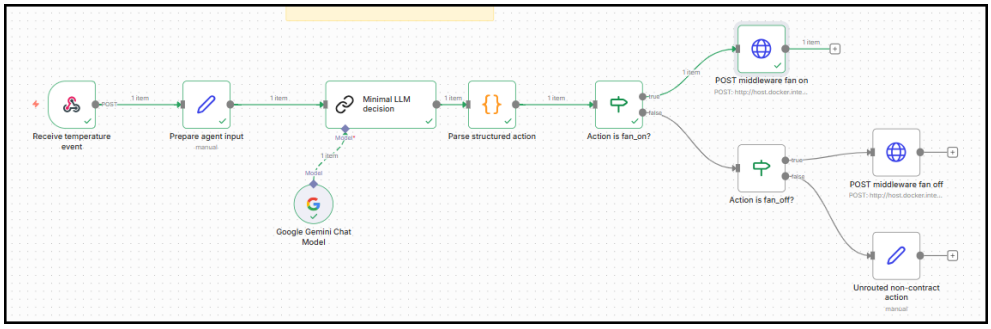

Figure 6: *Minimal agent-enhanced n8n workflow canvas.*

5.7 Safety and Validation Design

The safety layer is documented as a minimum design and specification layer rather than implemented as a measured runtime enforcement layer. It consists of an action policy, an output validation design, a human-in-the-loop approval design, and three documented example cases. The action policy, parser and validation design, and approval design are listed with the other source artifacts in Appendix A [13].

The safety design is based on the shared action contract. Under the documented policy, an action would be allowed to proceed without human approval only when it parses as JSON, matches the action schema, uses `fan_on` or `fan_off`, targets `fan_1`, contains a non-empty reason, has no unexpected fields, and sets `requires_approval` to `false`. A malformed action, an action with missing fields, an unknown action, an unknown target, or an action with unexpected fields would be blocked by the policy. A risky or unknown action may be shown for human review as evidence, but policy version 1 does not allow an unknown action or target to be approved into a middleware call.

The repository includes one allowed case, one blocked malformed case, and one risky/unknown case. The allowed case describes a valid `fan_on` action targeting `fan_1`. The blocked case describes an incomplete action missing required fields. The risky case describes an invented `open_window` action targeting an unknown device, which is outside the thesis scenario and must not reach the fan endpoints.

This safety layer matters because it defines the boundary that workflow and agent output should satisfy before action execution. How-

ever, it is not a production-grade runtime safety framework and it is not evidence that all unsafe actions are blocked by an implemented runtime gate. In the final implementation, it should be interpreted as the minimum validation and approval design required to make the contract boundary explicit and discussable in the report. The measured workflow evidence supports contract-shaped output and routing for the defined cases, while full runtime schema validation and safety enforcement remain future work.

5.8 Evaluation Harness

The evaluation harness is implemented through the evaluation data files and supporting scripts. Its purpose is to make the proof-of-concept measurable without adding a heavy observability stack. The harness uses standard-library Python scripts and CSV files so that test execution and aggregation remain lightweight and transparent [19], [20].

The test cases include deterministic workflow cases, agent workflow cases, and safety-mode cases. The metric fields are documented in the evaluation metrics schema. Raw result files and processed summaries are written to the evaluation result folders listed in Appendix A.

The collection script posts the selected test payloads to a supplied n8n webhook URL. For workflow modes, it records the mode, test case identifier, run index, input temperature, expected action, observed action, HTTP status, success flag, latency, observed response, and error/notes fields. It also attempts to collect CPU and RAM values through Docker statistics when available [18], and it records thermal data only when a supported thermal source exists. The aggregation script reads raw CSV files and writes processed summaries for latency, resources, safety outcomes, and baseline-versus-agent comparison.

Chapter 6 reports the numerical outcomes from these CSV files. This implementation chapter describes the harness structure and the measured fields, while final latency, resource, and correctness values are presented in the Results chapter.

5.9 Raspberry Pi Middleware/Action-Endpoint Validation Setup

The Raspberry Pi work was implemented as middleware/action-endpoint validation rather than full deployment. In this setup, n8n stayed on the PC and continued to run inside Docker. The Raspberry Pi ran only the Python middleware/action endpoint side. The middleware was started on the Pi with the host bound to `0.0.0.0`, allowing the PC-hosted n8n workflow to call the Pi over the network using HTTP [6].

The validation reused the same temperature-to-fan scenario. High-temperature and low-temperature cases were sent through the PC-hosted n8n workflow, and the workflow called the Raspberry Pi middleware endpoints for `/fan/on` and `/fan/off`. The Raspberry Pi validation evidence includes the final workflow-run CSV, notes, startup/IP transcript, canvas and output screenshots, direct endpoint latency evidence, CPU/RAM evidence, and thermal evidence.

This split validation strengthens the edge-computing angle of the thesis without changing the core architecture. It shows that the action endpoint side of the system can be moved from the PC to the Raspberry Pi while preserving the PC-hosted workflow engine. At the same time, its limitations are important: n8n was not deployed on the Raspberry Pi, the whole system was not moved to the Pi, the fan action remained simulated, and no real GPIO hardware was controlled.

5.10 Summary of Implementation

The implemented system forms one complete proof-of-concept pipeline. A temperature event can be sent into a PC-hosted n8n workflow, evaluated either by a deterministic threshold path or by a minimal agent-enhanced path, represented through a shared JSON action contract, scoped by a documented validation and approval design, executed through Python middleware as a simulated fan action, and recorded through a CSV-based evaluation harness. Raspberry Pi validation extends this pipeline by moving the middleware/action endpoint to the Pi while keeping n8n on the PC. This focused implementation is complete enough to support the thesis evaluation of action correctness, contract-shaped agent output, latency, resource behavior, and deployment limitations.

6 Results

This chapter reports the observed results from the defined proof-of-concept. The chapter is limited to factual reporting of the available evidence. Interpretation of what the results mean for the research questions is left to Chapter 7.

6.1 Overview of Evaluated System

The results cover the evaluated workflow modes, documented validation cases, CSV outputs, and Raspberry Pi middleware validation. The local workflow tests used the same two temperature cases defined in the evaluation dataset. These values were chosen on opposite sides of the deterministic threshold of `30.0 C`: the high-temperature case `31.4 C` is above the threshold and is expected to produce `fan_on`, while the low-temperature case `24.5 C` is below the threshold and is expected to produce `fan_off`.

The reported latency values are single-run observations from the final successful run for each defined case. They are used to show that the workflow-to-action path was measurable in the implemented proof-of-concept, not to provide a statistical benchmark. Since the evaluation did not include repeated latency trials per case, no standard deviation is reported. For this reason, latency values in the result tables are rounded to approximately two or three significant figures.

For the agent-enhanced workflow, the results use the term *contract-shaped* rather than *runtime schema-validated*. This means that the observed output contained the expected action fields and supported fan-action values and could be parsed and routed by the workflow for the defined cases. The results do not include runtime JSON Schema validation logs or measured safety-enforcement outcomes.

The local workflow results are recorded in raw CSV files, and the processed summaries are stored in the processed result files. Raspberry Pi validation evidence is stored with the Raspberry Pi result artifacts. Full repository paths for these evidence files are listed in Appendix A. Additional screenshots and raw validation evidence are listed in Appendix A.

6.2 Deterministic Baseline Results

The deterministic baseline applies the fixed threshold rule documented in the workflow evidence: temperatures greater than or equal to `30.0 C` are routed to `fan_on`, and lower temperatures are routed to `fan_off`. The reported input values `31.4 C` and `24.5 C` are therefore branch-test inputs, not alternative threshold values. The Step 6 runtime verification notes report that both required branches were verified locally

through the n8n test webhook. The final raw deterministic CSV contains one successful row for each defined deterministic test case. Table 3 summarizes the deterministic baseline results for the two defined temperature cases. The source file is listed in Appendix A.

Table 3: Deterministic baseline results.

Test case	Input (C)	Expected	Observed	HTTP	Approx. latency (ms)
deterministic_high_temp	31.4	fan_on	fan_on	200	480
deterministic_low_temp	24.5	fan_off	fan_off	200	177

Both deterministic CSV rows have success=true. The observed middleware responses also identify the fan action as simulated. For the high-temperature case, the response contains action=fan_on and fan=on. For the low-temperature case, the response contains action=fan_off and fan=off. These rows are the final successful deterministic raw result rows. Each latency value in Table 3 represents one final successful run for the corresponding deterministic test case. A previous deterministic production-webhook setup attempt returned HTTP 404; that file is treated as setup evidence rather than final action-correctness evidence.

6.3 Agent-Enhanced Workflow Results

The agent-enhanced workflow was evaluated with the same high- and low-temperature inputs. The Step 7 runtime verification evidence reports that the workflow was imported into n8n, connected to a Google Gemini Chat Model, and tested with both defined temperature cases. The evidence includes screenshots for the agent output, parsed action, and resulting middleware fan action. The workflow was configured for stateless execution with no memory node.

For the high-temperature case, the evidence reports that the agent selected fan_on, produced structured action output containing action_id=fan_on, and routed the parsed action to the /fan/on middleware endpoint. For the low-temperature case, the evidence reports that the agent selected fan_off, produced structured action output containing action_id=fan_off, and routed the parsed action to the /fan/off middleware endpoint. The raw agent CSV records the observed mid-

dleware actions and response status for these two cases. Table 4 summarizes the agent-enhanced workflow results for the same two input cases. The source file is listed in Appendix A.

Table 4: Agent-enhanced workflow results.

Test case	Input (C)	Expected	Observed	HTTP	Approx. latency (ms)
`agent_high_temp`	31.4	`fan_on`	`fan_on`	200	2,670
`agent_low_temp`	24.5	`fan_off`	`fan_off`	200	2,000

Both agent CSV rows have `success=true`. The CSV file records the final observed middleware action, while the screenshot evidence records the structured agent output and parsed action routing. Each latency value in Table 4 represents one final successful run for the corresponding agent-enhanced test case. Together, the evidence supports the claim that the agent workflow produced contract-shaped `fan_on` and `fan_off` actions for the defined cases and that these supported actions were parsed and routed through the workflow. This should not be read as evidence of full runtime JSON Schema validation. The evidence is limited to this workflow setup and is not a model comparison or multi-agent evaluation.

6.4 Safety and Validation Case Results

The safety and validation evidence consists of documented cases and expected policy outcomes. The safety-case files are listed in Appendix A. The Step 8 policy defines the allowed actions as `fan_on` and `fan_off`, the allowed target as `fan_1`, and the required fields as `action_id`, `target`, `reason`, and `requires_approval`.

The documented safety and validation cases are summarized in Table 5.

Table 5: Documented safety and validation cases from Step 8.

Case	Input summary	Expected policy outcome	Middleware endpoint
Allowed	Complete `fan_on` action for `fan_1`; `requires_approval=false`.	Allow after validation.	`POST /fan/on`
Blocked	`fan_on` action missing required fields such as `reason` and `requires_approval`.	Block.	None
Risky / approval	`open_window` action targeting `unknown_device`; `requires_approval=true`.	Reject for direct execution; may be shown for review.	None

The safety result is based on documented validation cases rather than runtime enforcement logs. The processed safety-outcome file contains only the header row, so the safety layer should be interpreted as a validation design and future implementation target rather than as measured runtime enforcement.

6.5 Evaluation Harness Outputs

The evaluation harness produced both raw CSV files and processed summary files. The raw CSV files contain one successful deterministic run for each deterministic case and one successful agent run for each agent case. A previous misconfigured deterministic webhook attempt is preserved as setup evidence and records HTTP `404` responses because the production webhook was not registered at the time of that setup attempt.

The evaluation output files used as report evidence are summarized in Table 6. The full repository paths for the raw and processed evidence files are listed in Appendix A.

Table 6: Evaluation harness output files.

Artifact	Observed content/status
Final deterministic raw CSV	Two successful local deterministic workflow rows: high temperature and low temperature.
Final agent raw CSV	Two successful local agent workflow rows: high temperature and low temperature.
Non-final setup evidence	Earlier HTTP `404` webhook setup evidence; not used as final action-correctness evidence.
Latency summary	Processed latency summary regenerated from the final raw workflow CSV files.
Resource summary	Processed resource summary with local CPU/RAM fields where available and no local thermal values.
Baseline versus agent summary	Processed baseline-versus-agent summary regenerated from the final raw workflow CSV files.
Safety outcome summary	Header only; no runtime safety outcome rows.

The processed summaries were regenerated from the final raw CSV files after setup-failure evidence had been separated from the final raw aggregation input. The processed deterministic and agent summaries therefore contain one successful row per defined local workflow test case. The earlier deterministic `404` setup attempt remains preserved as non-final setup evidence, but it is not part of the regenerated final processed summaries.

For the local PC runs, CPU and RAM fields were available in the raw and processed CSV files through best-effort Docker statistics. Local thermal values were recorded as `not_available`, which matches the metrics schema expectation for the Windows/PC setup.

Table 7 combines the final full-workflow latency observations from the local deterministic results in Table 3, the local agent-enhanced results in Table 4, and the Raspberry Pi validation results in Table 8. The values are approximate single-run observations from the final successful run for each defined case. Direct Raspberry Pi endpoint latency is supporting evidence only and is not mixed with the full workflow latency values.

Table 7: Final workflow latency summary.

Source	Test case	Input (C)	Observed	Success	Approx. latency (ms)
Local deterministic	High temperature	31.4	`fan_on`	`true`	480
Local deterministic	Low temperature	24.5	`fan_off`	`true`	177
Local agent	High temperature	31.4	`fan_on`	`true`	2,670
Local agent	Low temperature	24.5	`fan_off`	`true`	2,000
Raspberry Pi validation	High temperature	31.4	`fan_on`	`true`	437
Raspberry Pi validation	Low temperature	24.5	`fan_off`	`true`	200

The internal CSV test identifiers are retained in the raw result files; the table uses human-readable labels for readability.

6.6 Raspberry Pi Middleware Validation Results

The Raspberry Pi validation used a split deployment. n8n stayed on the PC, while the Raspberry Pi ran the Python middleware/action endpoint. The middleware was started on the Raspberry Pi with the host bound to `MIDDLEWARE_HOST=0.0.0.0`. The PC-hosted n8n production webhook then called the Raspberry Pi middleware over HTTP.

The main full-workflow Pi validation evidence is stored in the Raspberry Pi validation CSV. It records two successful workflow calls: one high-temperature case and one low-temperature case. Table 8 summarizes the detailed Raspberry Pi middleware/action-endpoint full-workflow validation rows.

Table 8: Raspberry Pi middleware/action-endpoint full-workflow validation.

Test case	Input (C)	Expected	Observed	HTTP	Approx. latency (ms)
`deterministic_high_temp`	31.4	`fan_on`	`fan_on`	200	437
`deterministic_low_temp`	24.5	`fan_off`	`fan_off`	200	200

These approximate latency values are full-workflow single-run observations for the PC-hosted n8n workflow calling the Raspberry Pi middleware. They are separate from direct middleware endpoint latency. The direct endpoint measurements are supporting evidence only because they do not include n8n webhook execution or workflow routing. The same two Raspberry Pi rows are also included in Table 7 and Figure 7 so they can be read together with the local deterministic and agent-enhanced latency observations.

The Raspberry Pi high-temperature full-workflow latency is slightly lower than the corresponding local deterministic latency. This should not be interpreted as evidence that the Raspberry Pi setup is generally faster. The latency values are single-run observations and can be affected by webhook execution, Docker state, network timing, and normal runtime variation. Repeated runs would be needed before making a stable performance comparison.

Figure 7 visualizes the combined full-workflow latency observations summarized in Table 7. The figure includes the local deterministic rows, the local agent-enhanced rows, and the Raspberry Pi validation rows. It is a visualization of single-run observations, not a statistical benchmark.

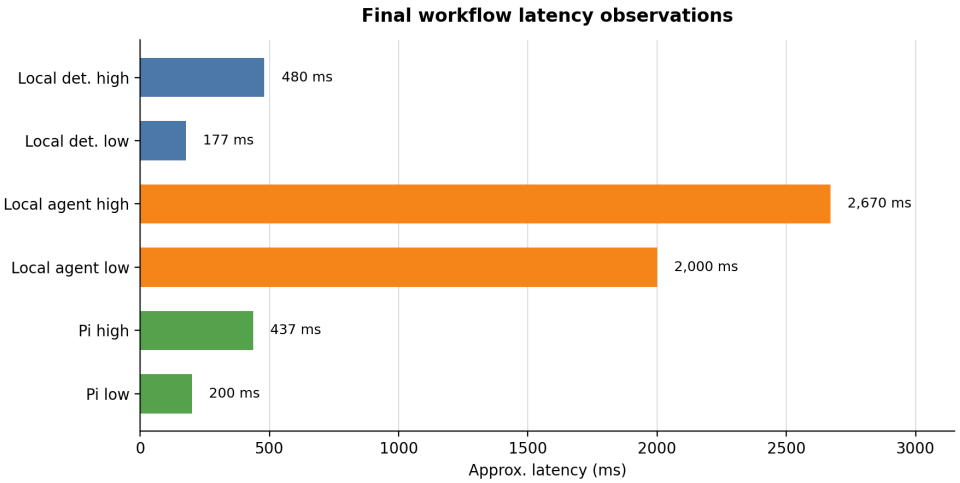

Figure 7: *Final workflow latency observations for local deterministic, local agent-enhanced, and Raspberry Pi validation runs.*

The observed middleware process was `python3 -m middleware.api.app`. The supporting endpoint and resource observations are summarized in Table 9.

Table 9: Supporting Raspberry Pi middleware observations.

Observation	Value	Interpretation
Direct `fan_on` endpoint latency	135.44 ms	Direct middleware endpoint call, not full workflow latency
Direct `fan_off` endpoint latency	233.27 ms	Direct middleware endpoint call, not full workflow latency
Middleware CPU usage	0.0 %	Instantaneous process observation
Middleware memory usage	0.5 %	Instantaneous process observation
Middleware RSS	21048 KB	Resident memory size of the middleware process
Calculated middleware RAM	20.95 MB	RSS converted to MB
Raspberry Pi temperature	49.6 C	Device thermal observation during validation

The full-workflow CSV retains the Step 9-style fields and records `th`

ermal_c=not_available. The Raspberry Pi thermal value of 49.6 C is documented separately in the Pi validation notes and thermal screenshot evidence, which uses the Raspberry Pi vcgencmd utility context [24]. Figure 8 shows the selected Raspberry Pi validation output screenshot for the high-temperature fan-on case.

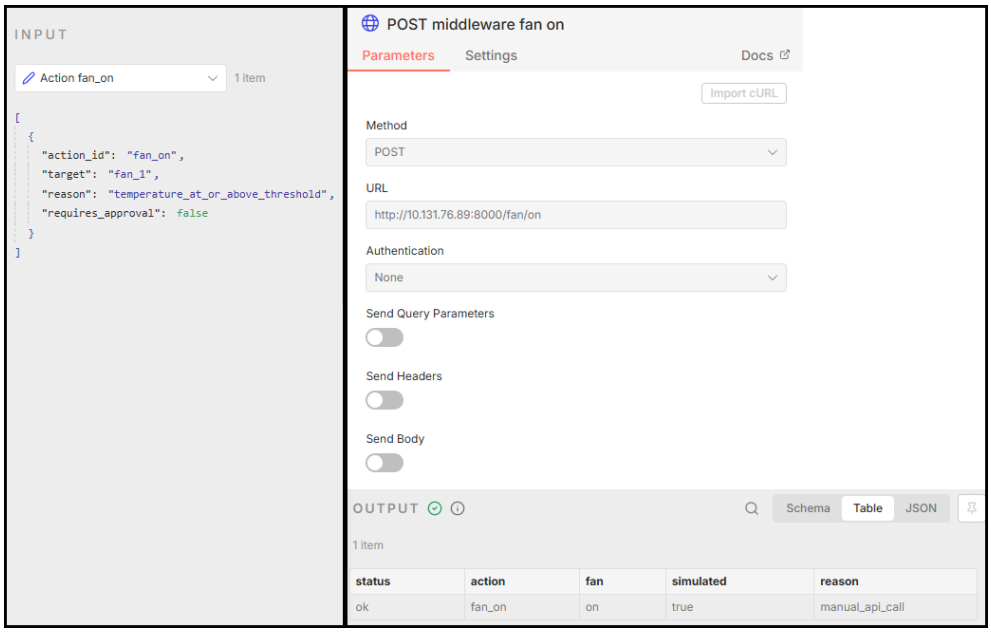

Figure 8: *Raspberry Pi middleware/action-endpoint validation output for the high-temperature fan-on case.*

The Pi validation result is not a full n8n deployment on the Raspberry Pi. It shows that the PC-hosted n8n workflow could call the Pi-hosted middleware/action endpoint for the defined high- and low-temperature cases. The fan actions remained simulated middleware actions, and no real GPIO hardware was controlled.

6.7 Summary of Results

The results show the behavior of the same evaluated system across local workflow execution, documented validation cases, and Raspberry Pi middleware validation. The deterministic baseline produced the expected fan_on and fan_off actions for the two defined temperature cases in the final raw deterministic CSV. The agent-enhanced workflow produced structured, contract-shaped action output for the same defined cases according to the Step 7 screenshot evidence, and the raw agent CSV records successful middleware responses.

The processed deterministic and agent summaries contain one successful row per defined local workflow test case, while the safety out-

come summary currently contains only headers. The Raspberry Pi middleware validation showed that PC-hosted n8n could call a Raspberry Pi-hosted middleware/action endpoint for the two deterministic temperature cases. The results are proof-of-concept evidence for the defined workflow-to-action scenario rather than broad benchmark results.

7 Discussion

This chapter interprets the results from Chapter 6 in relation to the research questions and the design choices described in Chapters 3–5. The discussion first states the main limitations, then maps the available evidence to the research questions, and finally discusses the project method, broader relevance, and ethical considerations. The chapter analyzes the evaluated workflow-to-action architecture rather than proposing a new implementation.

7.1 Limitations of the Current Implementation

The evaluation is limited by its scale. The system was evaluated with two defined temperature cases and one recorded final run per case, which is enough to check the implemented proof-of-concept but not enough for statistical reliability claims. The results are evidence for the defined workflow-to-action scenario rather than broad benchmarking.

The implementation also uses simulated fan actions. The results describe workflow behavior, contract-shaped action output, middleware routing, latency measurement, and split deployment behavior. They do not evaluate electrical reliability or physical actuator behavior, which would require sensor calibration, GPIO control, and hardware safety testing.

The Raspberry Pi validation has a specific role in the thesis. It shows that the middleware/action endpoint can run on the Raspberry Pi and be reached from the PC-hosted workflow over the network. Running the full n8n runtime on Raspberry Pi would introduce a separate deployment and resource question and is therefore treated as future work.

The safety layer is evaluated as a documented validation design with allowed, blocked, and risky cases. It supports discussion of the contract and validation boundary, while runtime enforcement would require additional implementation and measurement.

Finally, the agent-enhanced workflow is intentionally focused. It uses one prompt setup, one configured model path, and stateless execution. The result should therefore be read as evidence for whether an LLM decision step can produce contract-shaped action output for the defined scenario, not as a general evaluation of LLM reliability or multi-agent behavior.

7.2 Evidence Mapping for the Research Questions

Table 10 summarizes how the available evidence answers the three research questions. The table is not a replacement for the detailed results in Chapter 6; it shows how the factual evidence is interpreted in this chapter.

Table 10: Mapping between research questions, evidence, results, and interpretation.

Research question	Evidence used	Result	Interpretation
RQ1	Deterministic workflow CSV rows and Table 3.	`31.4 C` produced `fan_on`; `24.5 C` produced `fan_off`.	The deterministic path behaved correctly for the defined cases and provides a transparent baseline.
RQ2	Agent workflow CSV rows, Step 7 screenshot evidence, and the shared action contract.	The agent path produced structured `fan_on` and `fan_off` action output for the same cases.	The agent step can fit into the same contract-shaped action pipeline, without implying broad agent autonomy.
RQ3	Local workflow CSVs, Table 7, Raspberry Pi workflow CSV, and supporting resource observations.	Local and Raspberry Pi middleware-validation runs recorded latency, action outcomes, and supporting CPU/RAM/thermal evidence.	The evidence supports the defined evaluation scope and shows that PC-hosted n8n can call a Pi-hosted middleware/action endpoint.

7.3 Discussion of RQ1: Deterministic Workflow Accuracy

RQ1 asks how accurately the deterministic n8n workflow produces the expected fan action for the defined temperature test cases. The deterministic baseline behaved as expected for the final high- and low-temperature cases reported in Table 3. The high-temperature input

was routed to `fan_on`, and the low-temperature input was routed to `fan_off`. The deterministic workflow is therefore correct within the defined evaluation scope and evaluated run count.

This result matters because the deterministic path is the reference point for the rest of the thesis. A threshold workflow is transparent: the rule is visible, the threshold is fixed, and the route from input value to action endpoint can be inspected. The deterministic path therefore verifies that the webhook path, workflow decision, middleware endpoints, and simulated fan responses can support the temperature-to-fan scenario before the agent-enhanced path is considered.

The evidence is still tied to the defined cases. The evaluation does not include boundary-value tests around exactly `30.0 C`, noisy input streams, malformed temperature events, long-duration operation, or concurrent workflow executions. Within the evaluated scenario, the deterministic workflow produced the expected actions; broader operational claims would require additional testing.

7.4 Discussion of RQ2: Agent-Enhanced Contract-Shaped Output

RQ2 asks how accurately the minimal agent-enhanced workflow produces contract-shaped action output compared with the deterministic baseline for the defined cases. In this report, *contract-shaped* means that the observed output contained the expected action fields and supported fan-action values and could be parsed and routed by the workflow. The agent-enhanced workflow produced the expected `fan_on` and `fan_off` middleware actions for the same two temperature cases, as reported in Table 4. The screenshot evidence also shows that the workflow produced structured action output and parsed the action before routing it to the middleware endpoint. This is different from proving full runtime JSON Schema validation.

The repository contains a JSON Schema for the expected action object, and the safety artifacts describe how validation and approval should work. However, the final evaluated implementation does not include a measured runtime schema-validation gate with validation pass/fail logs. The evidence for RQ2 should therefore be read as evidence of contract-shaped output and workflow-level parsing/routing for the defined cases, not as evidence of production-grade runtime contract enforcement.

The agent path is useful because it tests whether an LLM decision step can be constrained to fit the same action pipeline as the deterministic baseline. In this temperature scenario, the deterministic rule is eas-

ier to explain and is sufficient for the action choice. The agent path therefore should not be interpreted as an attempt to outperform the deterministic rule. Its role is to show that generated output can be represented as an action object with fields such as `action_id`, `target`, `reason`, and `requires_approval`, then parsed and routed through the same middleware boundary.

This interpretation connects the thesis to work on language-model-based reasoning and action [10]. In this thesis, that idea is applied in a focused workflow setting: the agent-enhanced path receives the same temperature cases as the deterministic baseline and is expected to return contract-shaped action output. The result is therefore mainly evidence about whether an LLM decision step can be integrated into the workflow-to-action pipeline while preserving the expected action structure.

The RQ2 evidence is based on one prompt setup, one configured model path, and the same two temperature cases used by the deterministic baseline. Within that defined setup, the agent-enhanced workflow produced structured action output for the implemented cases and routed the expected fan actions through the middleware boundary. Because both paths were evaluated on two straightforward temperature cases, the evidence does not show a meaningful capability difference beyond output structure and latency observations.

7.5 Discussion of RQ3: Latency, Resource, and Deployment Behavior

RQ3 asks what latency, resource, and deployment behavior is observed during local execution and Raspberry Pi middleware validation. The local evaluation harness produced raw CSV evidence for deterministic and agent-enhanced workflow runs, and the Raspberry Pi validation produced full-workflow evidence for the PC-hosted n8n workflow calling the Pi-hosted middleware endpoint. These results provide inspectable measurement artifacts rather than relying only on screenshots or manual notes.

The distinction between full workflow latency and direct endpoint latency is central to interpreting the Raspberry Pi evidence. The full workflow values include the PC-hosted n8n workflow path and the HTTP call to the middleware endpoint. The direct Raspberry Pi endpoint values measure only middleware response behavior and do not include n8n webhook execution or workflow routing. For this reason, the direct endpoint values are useful supporting evidence, but they cannot replace the full workflow values in Table 8.

The CPU, RAM, and thermal observations should be treated as deployment observations rather than performance benchmarks. Local PC resource values were collected through best-effort Docker statistics where available, while local thermal values were recorded as `not_available`. The Raspberry Pi evidence includes middleware process CPU/RAM observations and a separately documented thermal value. These observations indicate that the evaluation harness can capture resource-related evidence, but the workload is too constrained for stable performance claims.

The deployment behavior still supports the edge-computing motivation of the thesis [3], [4]. The Raspberry Pi validation demonstrates that the action endpoint side can run on separate hardware while n8n remains on the PC. This is the split architecture implemented by the thesis and should be interpreted as Raspberry Pi middleware/action-endpoint validation.

7.6 Project and Work Method Discussion

The PC-first n8n setup was a practical and methodological choice. Running n8n locally on the PC through Docker gave a stable workflow environment and reduced the risk that the thesis would become primarily about platform deployment. This made it possible to focus the evaluation on the event-to-action path, the middleware boundary, the contract-shaped action output, and the CSV evidence.

Raspberry Pi validation was focused on the middleware/action endpoint side for the same reason. Moving the Python endpoint to the Raspberry Pi tested whether the workflow could call an edge-side action boundary over HTTP, while keeping the workflow engine itself in the stable PC-hosted environment. This choice gave the thesis hardware-side validation without changing the main research question into a Raspberry Pi optimization study.

The comparison between a deterministic baseline and a minimal agent-enhanced workflow was also defensible for the defined evaluation scope. The deterministic workflow established a transparent reference behavior. The agent-enhanced workflow then tested whether an LLM decision step could produce action output that still fit the same contract and middleware path. Because both paths used the same action endpoints and temperature cases, the comparison stayed focused on the decision step and output structure rather than on unrelated deployment differences.

The work method also depended on traceable evidence. Report notes, audit passes, CSV output, screenshot evidence, and explicit scope boundaries helped keep the thesis aligned with the repository artifacts. Fi-

nalizing the implementation state before report writing was necessary because the report could then describe a stable evidence base instead of chasing changing workflows, scripts, or result files. As a result, thesis claims are tied to artifacts that can be inspected.

7.7 Role of Middleware, Contracts, and Validation

The middleware boundary is one of the central design choices in the thesis. Without it, workflow or agent output could be treated as direct action execution. By placing Python middleware between the workflow and the simulated fan action, the architecture creates a controlled place where action requests can be represented, inspected, routed, and later replaced with hardware-specific behavior if the scope is extended.

The JSON contracts support this boundary by reducing ambiguity. A workflow that emits loose text is difficult to evaluate objectively because the report must infer whether the text means `fan_on`, `fan_off`, or something else. A contract-shaped action object gives the system explicit fields and allowed values. JSON Schema is therefore useful because it gives the report a precise structure for discussing action validity [7].

The documented validation design adds a safety-oriented interpretation of the same boundary. It distinguishes allowed fan actions, malformed actions, and risky or unknown actions. LLM output should not be passed directly to an action endpoint without checking what it contains. In this thesis, the validation boundary separates generated workflow output from middleware execution. If an output is unclear, risky, or outside the approved action set, it should be reviewed before any action is carried out [13].

In this implementation, the validation boundary defines the expected behavior for action outputs before they reach the middleware. The safety cases describe which outputs should be accepted, blocked, or held back from direct execution. They do not show measured runtime enforcement. This gives the implementation a clear validation target, while stronger runtime enforcement, including JSON Schema validation with logged pass/fail outcomes, can be added and measured in future work.

7.8 Broader Relevance and Possible Use Cases

The fan example is a controlled proxy for a broader event-validation-action pattern. An event is received, a workflow selects or produces an action, the action is represented through a shared contract, a validation boundary limits what may proceed, middleware executes the action,

and the evaluation harness records evidence. The engineering value of the thesis lies in this pattern rather than in the fan action by itself.

In smart building control, a similar pattern could be used for ventilation, heating, lighting, or occupancy-related actions [14]. In industrial monitoring, it could support alerts or controlled responses to temperature, vibration, or process-state events [15]. In environmental sensing, the same architecture could be adapted to air quality, humidity, water-level, or greenhouse monitoring. In health and environment monitoring contexts, the pattern could support event-driven notifications or approved actions where traceability and validation are important.

These examples indicate possible application areas, while the present evaluation remains focused on the temperature-to-fan scenario. Further work would be required before applying the architecture to physical actuation or operational environments.

7.9 Ethical and Societal Considerations

Automated and AI-assisted workflows become more sensitive when their outputs can trigger physical actions. Even a fan example illustrates the issue: once a workflow decision becomes an action request, the system needs clear boundaries around what is allowed to happen. This concern grows in domains such as buildings, industrial systems, environmental monitoring, or health-related contexts, where incorrect actions may affect comfort, equipment, safety, or trust.

For that reason, validation before execution is both a technical and ethical design concern. The system should make it possible to inspect what action was requested, why it was requested, whether it matched an approved contract, and whether it should require human review. The documented validation design and human-approval case point toward this responsibility: risky or unknown outputs should not be executed directly simply because a workflow or agent produced them.

The report therefore relies on saved evidence rather than memory or assumptions. The CSV files show the measured runs, the screenshots show the workflow behavior, and the contracts and safety cases show the rules the system was built around. This makes the work easier to check, repeat, and build on later.

8 Conclusions

8.1 Answers to Research Questions

RQ1 asked how accurately the deterministic n8n workflow produced the expected fan action for the defined temperature cases. The deterministic workflow produced the expected result for both tested cases: the high-temperature case led to `fan_on`, and the low-temperature case led to `fan_off`. This evidence supports the deterministic baseline for the defined scenario and run count reported in Chapter 6.

RQ2 asked how accurately the minimal agent-enhanced workflow produced contract-shaped action output compared with the deterministic baseline for the defined cases. The agent-enhanced workflow produced structured output with the expected action fields and supported fan-action values for the same defined cases, and the resulting actions were parsed and routed through the same middleware boundary. This result shows that an LLM decision step can be integrated into the workflow-to-action pipeline while preserving the expected action structure for the implemented cases. It does not show full runtime JSON Schema validation or production-grade safety enforcement.

RQ3 asked what latency, resource, and deployment behavior was observed during local execution and Raspberry Pi middleware validation. The CSV-based evaluation captured local workflow outcomes and latency evidence, while the Raspberry Pi validation showed that PC-hosted n8n could call the Pi-hosted middleware/action endpoint and receive the expected simulated fan responses. Because the latency values are based on single runs, they should be treated as proof-of-concept observations rather than stable benchmark measurements. The resource and thermal observations provide supporting evidence for this defined deployment pattern, but not a large-scale benchmark or full n8n-on-Pi result.

8.2 Final Conclusion

The thesis demonstrates a focused n8n-based workflow-to-action architecture for the temperature-to-fan scenario. Its contribution is the evaluated integration of workflow orchestration, middleware-based action execution, structured contracts, documented validation cases, CSV-based measurement, and Raspberry Pi middleware validation into one measurable proof-of-concept. The work shows that a workflow-to-action system can be made structured, measurable, and clear enough to evaluate when its contract boundaries and implementation scope are kept explicit. The Raspberry Pi validation strengthens the edge-computing part of the architecture by showing that the middleware/action endpoint can run on separate hardware and be called over the network

from the PC-hosted workflow.

8.3 Future Work

Future work could extend the proof-of-concept by deploying the full n8n stack on Raspberry Pi or another edge device and comparing that deployment with the PC-hosted workflow used in this thesis. Another natural extension would be to replace the simulated `fan_on` and `fan_off` endpoints with GPIO-connected sensor and actuator hardware, while preserving the middleware and contract boundary.

The evaluation could also be expanded with more temperature cases, repeated runs, and clearer visual summaries of latency and resource behavior. The current validation design could be developed into stronger runtime safety enforcement, including actual JSON Schema validation during workflow or middleware execution, logged validation pass/fail outcomes, clearer approval handling, and more explicit rejection of unknown or risky actions. The agent-enhanced workflow could be evaluated with additional prompts, models, and memory strategies, provided that such work remains grounded in measurable action outputs rather than broad claims about autonomy.

Finally, the architecture could be adapted to more realistic IoT contexts, such as smart building control, industrial monitoring, environmental sensing, or health and environment monitoring. These extensions would need their own validation and safety analysis, but they follow naturally from the same pattern used here: receive an event, convert it into a structured action decision, validate the action boundary, execute through middleware, and record evidence.

References

[1] n8n. "N8n docs," Accessed: May 24, 2026. [Online]. Available: https://docs.n8n.io/

[2] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow patterns," *Distributed and Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003. DOI: 10.1023/A:1022883727209

[3] L. Atzori, A. Iera, and G. Morabito, "The internet of things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010. DOI: 10.1016/j.comnet.2010.05.010

[4] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017. DOI: 10.1109/MC.2017.9

[5] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019. DOI: 10.1109/JPROC.2019.2918951

[6] Raspberry Pi Ltd. "Raspberry pi documentation," Accessed: May 24, 2026. [Online]. Available: https://www.raspberrypi.com/documentation/

[7] JSON Schema. "Json schema specification," Accessed: May 24, 2026. [Online]. Available: https://json-schema.org/specification

[8] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. Accessed: May 24, 2026. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

[9] R. T. Fielding, M. Nottingham, and J. Reschke, *Http semantics*, RFC 9110, 2022. DOI: 10.17487/RFC9110 Accessed: May 24, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[10] S. Yao et al., "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. Accessed: May 24, 2026. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[11] S. Hosseini and H. Seilani, "The role of agentic ai in shaping a smart future: A systematic review," *Array*, vol. 26, p. 100 399, 2025. DOI: 10.1016/j.array.2025.100399

[12] R. Sapkota, K. I. Roumeliotis, and M. Karkee, *Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges*, arXiv preprint arXiv:2505.10468, 2025. arXiv: 2505.10468 [cs.AI]. Accessed: May 27, 2026. [Online]. Available: https://arxiv.org/abs/2505.10468

[13] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, "Power to the people: The role of humans in interactive machine learning," *AI Magazine*, vol. 35, no. 4, pp. 105–120, 2014. DOI: 10.1609/aimag.v35i4.2513

[14] D. Minoli, K. Sohraby, and B. Occhiogrosso, "Iot considerations, requirements, and architectures for smart buildings: Energy optimization and next-generation building management systems," *IEEE Internet of Things Journal*, vol. 4, no. 1, pp. 269–283, 2017. DOI: 10.1109/JIOT.2017.2647881

[15] H. Xu, W. Yu, D. W. Griffith, and N. Golmie, "A survey on industrial internet of things: A cyber-physical systems perspective," *IEEE Access*, vol. 6, pp. 78 238–78 259, 2018. DOI: 10.1109/ACCESS.2018.2884906

[16] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. DOI: 10.2307/25148625

[17] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302

[18] Docker Inc. "Docker container stats," Accessed: May 24, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/stats/

[19] Y. Shafranovich, *Common format and mime type for comma-separated values (csv) files*, RFC 4180, 2005. DOI: 10.17487/RFC4180 Accessed: May 24, 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4180

[20] Python Software Foundation. "Csv – csv file reading and writing," Accessed: May 24, 2026. [Online]. Available: https://docs.python.org/3/library/csv.html

[21] R. Morabito, "Virtualization on internet of things edge devices with container technologies: A performance evaluation," *IEEE Access*, vol. 5, pp. 8835–8850, 2017. DOI: 10.1109/ACCESS.2017.2704444

[22] Docker Inc. “Docker docs,” Accessed: May 24, 2026. [Online]. Available: https://docs.docker.com/

[23] Python Software Foundation. “Http.server – http servers,” Accessed: May 24, 2026. [Online]. Available: https://docs.python.org/3/library/http.server.html

[24] Raspberry Pi Ltd. “Raspberry pi os: Vcgencmd,” Accessed: May 24, 2026. [Online]. Available: https://www.raspberrypi.com/documentation/computers/os.html#vcgencmd

A Source Code and Artifacts

This appendix lists the main source-code files, workflow exports, evaluation files, and supporting evidence used in the thesis. The full project repository is available at:

`https://github.com/Rumple12/new-yacoub-thesis`

The appendix does not reproduce source code in full. Instead, it points to the repository artifacts needed to inspect, repeat, or extend the work.

A.1 Implementation Files

The main implementation files are listed below.

Table 11: Main implementation artifacts.

Path	Purpose
`infrastructure/docker/docker-compose.yml`	Docker Compose setup for the local n8n runtime.
`middleware/api/routes.py`	Python middleware HTTP routes for status and simulated fan actions.
`middleware/gpio/mock_sensor.py`	Simulated sensor and fan-state logic.
`scripts/collect_metrics.py`	Script used to collect workflow result rows.
`scripts/aggregate_results.py`	Script used to regenerate processed result summaries.

A.2 Workflow, Prompt, and Contract Artifacts

The workflow exports, prompt files, and shared contracts are listed below.

Table 12: Workflow, prompt, and contract artifacts.

Path	Purpose
`cognitive_logic/workflows/deterministic-baseline.json`	Exported deterministic baseline n8n workflow.
`cognitive_logic/workflows/deterministic-baseline.md`	Description of the deterministic workflow.
`cognitive_logic/workflows/agent-minimal.json`	Exported agent-enhanced n8n workflow.
`cognitive_logic/workflows/agent-minimal.md`	Description of the agent-enhanced workflow.
`cognitive_logic/prompts/system-prompt-v1.md`	Prompt used by the agent-enhanced workflow.
`cognitive_logic/memory/memory-choice-v1.md`	Documented memory choice for the agent-enhanced workflow.
`shared_interfaces/json-schema/sensor-event.schema.json`	JSON schema for sensor-style temperature events.
`shared_interfaces/json-schema/agent-action.schema.json`	JSON schema for contract-shaped action output.
`shared_interfaces/examples/`	Example sensor and action payloads.

A.3 Safety and Validation Artifacts

The validation and safety-layer documents are listed below. These files define the documented validation cases used in the report.

Table 13: Safety and validation artifacts.

Path	Purpose
`safety_layer/policies/action-policy-v1.md`	Action policy for allowed and unsupported action outputs.
`safety_layer/parsers/output-validation-v1.md`	Documented parser and validation design.
`safety_layer/approvals/hitl-v1.md`	Human-in-the-loop approval design.
`safety_layer/examples/`	Allowed, blocked, and risky validation examples.
`evaluation/datasets/test-cases.json`	Defined workflow and safety test cases.

A.4 Evaluation Result Files

The main raw and processed result files used in the report are listed below.

Table 14: Evaluation result artifacts.

Path	Purpose
`evaluation/results/raw/run_20260429T165733Z_deterministic.csv`	Final deterministic workflow result rows.
`evaluation/results/raw/run_20260429T165859Z_agent.csv`	Final agent-enhanced workflow result rows.
`evaluation/results/raw/dump/run_20260429T165702Z_deterministic.csv`	Non-final setup evidence from an earlier HTTP 404 webhook attempt.
`evaluation/results/processed/summary_latency.csv`	Processed latency summary.
`evaluation/results/processed/summary_resources.csv`	Processed resource summary.
`evaluation/results/processed/baseline_vs_agent.csv`	Processed deterministic-versus-agent summary.
`evaluation/results/processed/safety_outcomes.csv`	Safety outcome summary file; header only in the final evidence.
`evaluation/results/pi-validation/pi-workflow-run-final.csv`	Raspberry Pi middleware/action-endpoint validation rows.
`evaluation/results/pi-validation/pi-validation-notes.md`	Raspberry Pi validation notes.

A.5 Figure and Screenshot Evidence

The main report uses selected diagrams and screenshots. Additional raw screenshots and supporting evidence are kept in the repository instead of being reproduced throughout the report.

Table 15: Figure and screenshot evidence locations.

Path	Purpose
`thesis/MiunThesisTemplate-master/MiunThesisTemplate-master/Figures/`	Figures and screenshots included in the thesis.
`docs/architecture/`	Mermaid sources and notes for architecture and theory diagrams.
`cognitive_logic/workflows/evidence/step-06/`	Deterministic workflow screenshot evidence.
`cognitive_logic/workflows/evidence/step-07/`	Agent-enhanced workflow screenshot evidence.
`evaluation/results/pi-validation/`	Raspberry Pi validation screenshots, notes, and CSV evidence.

Figure 9 shows one example of local runtime evidence. Other raw screenshots are listed by folder above rather than inserted individually.

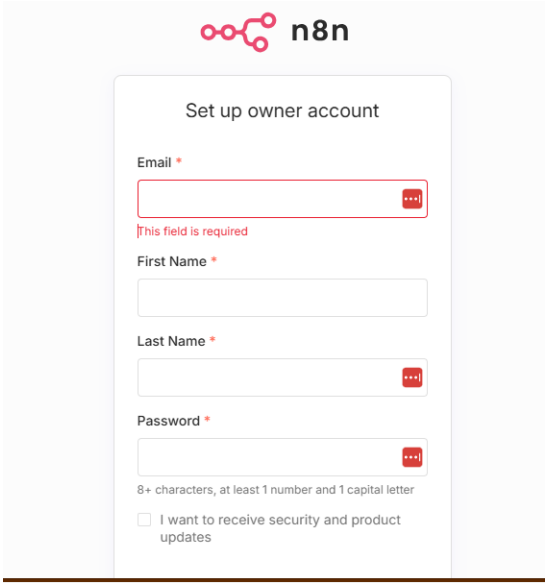

Figure 9: *Local n8n runtime verification at localhost port 5678.*

Figure 10 shows the detailed architecture diagram kept as supporting evidence.

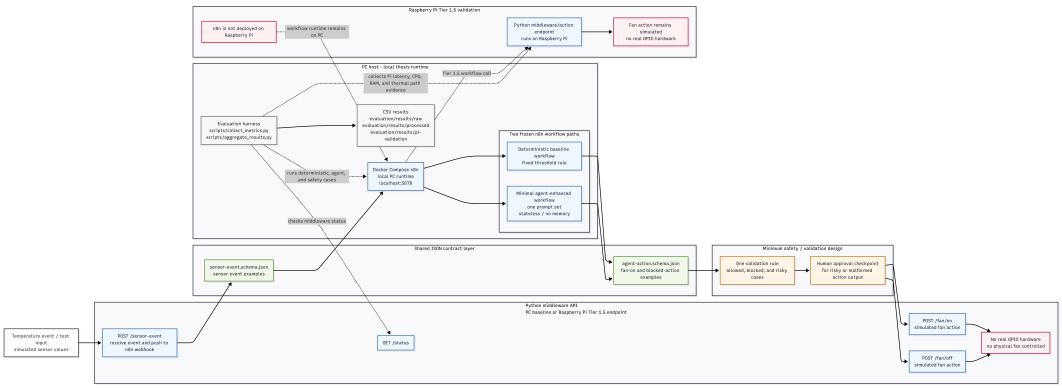

Figure 10: *Detailed architecture diagram preserved as supporting evidence.*

Figure 11 shows the Raspberry Pi middleware CPU and RAM observation used as supporting resource evidence.

```
yacoub@new-yacoub-pi:~ $ pgrep -af "middleware.api.app"
1842 python3 -m middleware.api.app
yacoub@new-yacoub-pi:~ $ PID=$(pgrep -f "middleware.api.app" | head -n 1)
yacoub@new-yacoub-pi:~ $ ps -p "$PID" -o pid,comm,%cpu,%mem,rss,args
    PID COMMAND         %CPU %MEM    RSS COMMAND
   1842 python3          0.0  0.5 21448 python3 -m middleware.api.app
yacoub@new-yacoub-pi:~ $ PID=$(pgrep -f "middleware.api.app" | head -n 1)
yacoub@new-yacoub-pi:~ $ RSS_KB=$(ps -p "$PID" -o rss=)
yacoub@new-yacoub-pi:~ $ echo "Middleware RAM MB: $(awk "BEGIN {printf \"%.2f\", $RSS_KB/1024}")"
Middleware RAM MB: 20.95
yacoub@new-yacoub-pi:~ $ PID=$(pgrep -f "middleware.api.app" | head -n 1)
yacoub@new-yacoub-pi:~ $ ps -p "$PID" -o %cpu=
 0.0
yacoub@new-yacoub-pi:~ $
```

Figure 11: *Raspberry Pi middleware CPU and RAM observation.*

Figures 12 and 13 show terminal evidence for the Raspberry Pi high- and low-temperature validation cases.

```
PS C:\Users\Jake_> Invoke-RestMethod -Method Post -Uri $piWebhookUrl -ContentType "application/json" -Body $highBody


status    : ok
action    : fan_on
fan       : on
simulated : True
reason    : manual_api_call
```

Figure 12: *Raspberry Pi terminal evidence for the high-temperature fan-on case.*

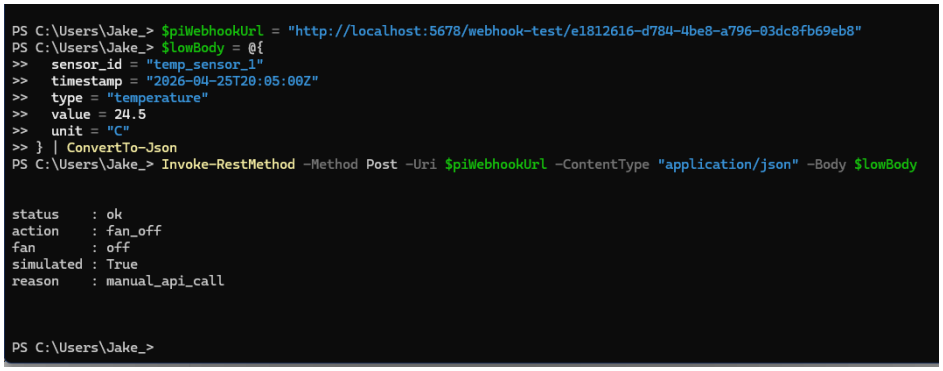

Figure 13: *Raspberry Pi terminal evidence for the low-temperature fan-off case.*